

SWE 4101

Object Oriented Concepts I

Prepared By:
Samia Nawsheen
Junior Lecturer, CSE



Objectives

01

Runtime Polymorphism

02

Inheritance



✓ Compile-Time Polymorphism (Recap)

- ✓ ● Also called Static Polymorphism / Early Binding
- ✓ ● Allows multiple methods with the same name to exist in a class, each with a different parameter list
- ✓ ● Helps us write cleaner, more efficient code



Runtime Polymorphism

- ✓ The method call is resolved at runtime rather than compile-time
- ✓ This is known as dynamic polymorphism or late binding
- ✓ At runtime, the JVM determines the actual object that the reference is pointing to and invokes the appropriate method

An Example

```
class Student {  
    private String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
}
```

```
Student s = new Student("John");
```

```
System.out.println(s);
```

← What will this print?



An Example

It prints something like:

Student@4eec7777

But wait...

We never wrote a method that tells Java how to print a Student

So where did this output come from?



An Example

- ✓ • `println` cannot literally print an object
- ✓ • The console can only display text
- ✓ • So Java needs a String representation of the object
- ✓ • `System.out.println(yourObject)`
- ✓ • By default, this statement will call the object's `toString()` method to obtain its text representation

An Example



- ✓ But we did not write a toString method in Student
- ✓ Java has a special built-in class called Object
- ~~*~~ Object is the **ultimate ancestor** of all Java classes
- ✓ Object provides some default methods, and one of those methods is toString()



✓ Method Overriding

If we want a customized output, we can write our own toString method

```
@Override  
public String toString() {  
    return "Student: " + name;  
}
```

The `@Override` annotation is often used to indicate that a method is being overridden

We are providing our own implementation of a method that is already defined - Method Overriding

Method Overriding

- ✓ Now there are two toString methods: `Object.toString()` and `Student.toString()`
- ✓ Which `toString()` method gets called?
- ✓ At runtime, JVM sees the object is actually a `Student`, so it uses `Student's` version



Method Overriding

```
Student s = new Student("John");  
System.out.println(s); // Student: John
```

The method call is resolved at runtime - hence Runtime
Polymorphism \



Inheritance



✓ Scenario: Company Payroll System

A company employs different types of employees, such as Software Engineers, Managers, and Interns

- ✓ Common attributes: Employee ID, Name, Salary
- ✓ Common behaviors: Display information, Receive salary
- ✓ Specialized behaviors: Engineers write code, managers conduct meetings and interns attend training



Solution

```
public class Engineer {  
    public String id;  
    public String name;  
    public double salary;  
  
    public Engineer(String id, String name, double salary) {  
        this.id = id;  
        this.name = name;  
        this.salary = salary;  
    }  
  
    public void displayInfo() {  
        System.out.println(id + " " + name);  
    }  
  
    public void receiveSalary() {  
        System.out.println("Salary paid");  
    }  
  
    public void writeCode() {  
        System.out.println("Writing code");  
    }  
}
```



Solution

```
public class Manager {  
    public String id;  
    public String name;  
    public double salary;  
  
    public Manager(String id, String name, double salary) {  
        this.id = id;  
        this.name = name;  
        this.salary = salary;  
    }  
  
    public void displayInfo() {  
        System.out.println(id + " " + name);  
    }  
  
    public void receiveSalary() {  
        System.out.println("Salary paid");  
    }  
  
    public void conductMeeting() {  
        System.out.println("Meeting started");  
    }  
}
```



Solution

```
public class Intern {  
    public String id;  
    public String name;  
    public double salary;  
  
    public Intern(String id, String name, double salary) {  
        this.id = id;  
        this.name = name;  
        this.salary = salary;  
    }  
  
    public void displayInfo() {  
        System.out.println(id + " " + name);  
    }  
  
    public void receiveSalary() {  
        System.out.println("Salary paid");  
    }  
  
    public void attendTraining() {  
        System.out.println("Training");  
    }  
}
```



✓ Question

- How many lines of code are identical?
- What happens if the company decides to add email to every employee?
- Update all three classes?
- Reasonable until there are 10/20/50 employee types
- Maintenance becomes a nightmare



✓ Question

- What if we create a single class that contains the common properties and behavior?
- And allow the other classes to reuse them instead of rewriting the same code?
- This is where inheritance comes to play



Extracting the Common Parts

```
public class Employee {
```

```
    public String id;  
    public String name;  
    public double salary;
```

```
    public Employee(String id, String name, double salary) {  
        this.id = id;  
        this.name = name;  
        this.salary = salary;  
    }
```

```
    public void displayInfo() {  
        System.out.println(id + " " + name);  
    }
```

```
    public void receiveSalary() {  
        System.out.println("Salary paid");  
    }
```

```
}
```

← Super/Base/Parent
Class



Extending Behavior

```
public class Engineer extends Employee {  
    public Engineer(String id, String name, double salary) {  
        super(id, name, salary);  
    }  
    public void writeCode() {  
        System.out.println("Writing code");  
    }  
}
```

← Sub/Derived/Child Class

- ✓ Engineer ***inherits*** everything from Employee
- ✓ It also has its own unique method - writeCode
- ✓ Duplication is further reduced by Constructor Chaining

★ ✓ Constructor Chaining (Recap)

✓ ● Calling one constructor from another constructor

✓ ● Reduces code redundancy

✓ ● You can chain constructors in two ways:

2 ways to call constructor:
i. `this()` - When inside the same class
ii. `super()` - When in a parent class to call it for a child class

✓ ○ **Within the Same Class:** Using the `this()` keyword to call overloaded constructors

✓ ○ **From a Base/Parent Class:** Using the `super()` keyword to call a parent class constructor during inheritance



Constructor Chaining Rules

- ✓ **First Line Rule:** The `this()` or `super()` call must be the very first statement in your constructor
- ✓ **Mutually Exclusive:** You cannot use both `this()` and `super()` inside the same constructor block
- ✓ **No Loops:** Recursive or cyclical constructor calls are strictly forbidden and cause compilation errors

Don't confuse Constructor Chaining with Constructor overloading!

Constructor Overloading - Creating multiple constructors with different numbers and types of parameters

Constructor Chaining - Calling another constructor from a constructor, with `this()` or `super()`



Extending Behavior

```
public class Manager extends Employee {  
  
    public Manager(String id, String name, double salary) {  
        super(id, name, salary);  
    }  
  
    public void conductMeeting() {  
        System.out.println("Meeting started");  
    }  
}
```

```
public class Intern extends Employee {  
  
    public Intern(String id, String name, double salary) {  
        super(id, name, salary);  
    }  
  
    public void attendTraining() {  
        System.out.println("Training");  
    }  
}
```

✱ Accessing Parent Functionality

- ✓ Employee is the **parent/super/base** class
- ✓ Engineer, Manager, Intern are the **child/sub/derived** classes
- ✓ Child classes can use methods and properties from Employee

```
Engineer e = new Engineer("E101", "Alice", 50000);
```

```
e.displayInfo(); // inherited method  
e.receiveSalary(); // inherited method  
e.writeCode(); // own method
```

```
System.out.println(e.salary); // inherited property
```

✱ ✓ Easier Maintenance

- ✓ ● If the company adds email, without inheritance, we would have to modify all three classes and every future employee type
- ✓ ● With inheritance, we can modify only Employee
- ✓ ● Every subclass immediately gains the feature

✓ When to use Inheritance

✓ When there is an **Is-A** relationship between classes

✓ Is an Engineer an Employee?

- Yes

✓ Is a Manager an Employee?

- Yes

✓ Is an Intern an Employee?

- Yes

Same goes for Animal, Cat and Dog.
-> Is Cat an animal? Answer: Yes
-> Is Dog an animal? Answer: Yes

So, Cat and Dog will inherit the parent class Animal.

```
public class Animal { }
```

```
public class Cat extends Animal{ }
```

```
public class Dog extends Animal{ }
```

✓ When NOT to use Inheritance

When the answer of Is-A relationship is false.

Class Engine

Class Car **extends** Engine

For example, Same Animal example, and we add Student class this time.

-> Is Student an animal? Answer: No

So, Student won't inherit Animal.

```
public class Animal { }
```

```
public class Student { } (Keep the class as it is)
```

✓ Is a car an engine?

○ No

✓ Therefore, inheritance does not make sense here



✓ Exercise

- Design a university system containing students, teachers and staff
- Think before answering:
 - What attributes are common to all three?
 - What behaviors are common to all three?
 - Which class could act as the parent class?
 - What unique features belong to each class?

public - accessible everywhere
default - inaccessible in different package
protected - inaccessible in different package non-subclass
private - inaccessible anywhere outside class, even in subclass of same package



Access Modifiers

Used to control the accessibility/visibility of classes, attributes, methods, constructors

	Default	Private	Protected	Public
Same Class	Yes	Yes	Yes	Yes
Same Package Subclass	Yes	No	Yes	Yes
Same Package Non-Subclass	Yes	No	Yes	Yes
Different Package Subclass	No	No	Yes	Yes
Different Package Non-Subclass	No	No	No	Yes